

Build-It Flow Handout — From `git push` to a Score

Part 3 of the briefing. How your submission is built, tested, and scored — and the environment it runs in.

a. You write the code

- All implementations in **C**.
- You **must use `dmalloc`** (`malloc-2.7.2.c`) as your allocator. It's intentionally insecure if used incorrectly — that's what makes Break-It interesting — so use it correctly.
- Every build must contain a `win()` function (the control-flow attack target).
- Do **not** obfuscate your source.

b. `make` builds it

Your code lives under `build/`. Running `make` in `build/` must produce an executable `stor`, **from source, with no Internet access** (committing binaries is not acceptable). You may edit the `Makefile`, but you **must keep all the compiler options**:

```
build:
  gcc -O0 -g -m32 -fno-stack-protector -o stor stor.c malloc-2.7.2.c

post:
  execstack --set-execstack stor
```

The flags are deliberate: **32-bit, no stack protector, executable stack**. They disable modern protections so that control-flow attacks are possible — that's the game. Keep them.

Can I change the `Makefile`? Yes — edit it freely to add your own source files and targets. What you must not change: the **mandated compiler flags must stay**, and `make` in `build/` must still produce `./`

stor from source, offline. Keeping the (deliberately weakening) flags is a contest rule — your source is published to other teams for Break-It — not an optional optimization.

c. What libraries are available

Your code is built and tested inside a **Docker container running Ubuntu 18.04 with a 32-bit toolchain**. Test locally in the same image — code that depends on a different OS/library version may behave differently in grading. **If a library isn't in the image, you can't use it.**

Toolchain: `build-essential · gcc · make · clang · llvm · cmake · gcc-multilib · execstack · python3.7 · openssl`

Libraries (64-bit, with `:i386` variants of the core ones for the 32-bit build):

Crypto / security	General
<code>libssl-dev</code> (OpenSSL)	<code>zlib1g-dev</code>
<code>libcrypto++-dev</code>	<code>libffi-dev</code>
<code>libsodium-dev</code> (+ <code>:i386</code>)	<code>uthash-dev</code>
<code>libmbedtls-dev</code>	<code>libjansson-dev</code> (JSON)
<code>libnacl-dev</code>	<code>libcppunit-dev</code>
<code>libgcrypt11-dev</code>	<code>libprotobuf-dev</code> , <code>libprotobuf-c0-dev</code> , <code>protobuf-compiler</code>
<code>libssh-dev</code>	

You don't have to roll your own crypto — a curated set of libraries is provided — but you must use it **correctly**. The 32-bit build is why `:i386` variants of the core libs (e.g. `libssl-dev:i386`, `libsodium-dev:i386`) are present.

Test locally in the real environment

The starter kit ships a `Dockerfile`. Build and run it to get a shell in an environment identical to grading:

```
docker build --tag bibifi .
docker run --rm -it bibifi
```

d. The build flow (what happens when you push)

1. `git push` → to your GitHub repo
2. Webhook fires → the contest is notified automatically
3. Runner picks it up → your commit is queued for grading
4. Container spins up → a fresh Ubuntu 18.04 sandbox (bibifi/bibifi-image)
5. Your code clones in → at that exact commit
6. ``make`` runs → in `build/`, producing `./stor`
7. Test suite runs → the functionality tests execute against your `stor`
8. Result → dashboard → status + per-test results recorded for your team

Each submission runs in its **own isolated, throwaway container**, so one team's code can't affect another's.

e. How the score is displayed

- The **dashboard** shows every submission's **status** and **per-test results**.
- **Build status:**
 - **Built** — compiled and passed all functionality tests.
 - **Build-fail** — didn't compile, or at least one functionality test failed.
- Passing Build-It **qualifies** your team: only qualified teams can be attacked in Break-It and can attack others.
- **Points** are earned in **Break-It** (see the Problem Statement & Scoring handouts): correctness 25, crash 50, and any security vulnerability (integrity / confidentiality / control-flow) 100. In the final standings a bug counts once and is shared among the teams that find it (the live scoreboard is provisional). See the Scoring handout.

Limits (or your submission times out)

Limit	Value
Per functionality test	60 seconds
Whole build + grade (session)	60 minutes
Break-It test input file size	600 MB

Common build pitfalls

- **Committed a binary / no `Makefile` in `build/`** → must build from source in `build/`.
- **Removed/changed compiler flags** → keep `-O0 -g -m32 -fno-stack-protector + execstack`.
- **Used a library not in the image** → only the libraries listed above are available.
- **Relied on your laptop's OS** → always test inside the Ubuntu 18.04 / 32-bit container.
- **Push didn't trigger anything** → check the webhook and that code is under `build/`.